

The background is a detailed still life painting. In the upper left, a lit candle sits in an ornate brass holder, with wax dripping down its sides. In the center, an open, antique book with yellowed, stained pages lies flat. A quill pen rests on the right page of the book. To the right of the book, a rolled-up parchment scroll is visible. The entire scene is set on a dark, textured wooden surface. The lighting is warm and focused, creating a scholarly atmosphere.

Logistic Regression & Regularization

Comprehensive Study Notes — Based on StatQuest Video Lectures & Hands-On ML with Scikit-Learn, Keras & TensorFlow (A. Geron)

MACHINE LEARNING FUNDAMENTALS SERIES

CLASSIFICATION & REGULARIZATION

Table of Contents

This guide covers the complete arc from probability fundamentals through model evaluation and regularization techniques.

Foundations

01

Odds, Log-Odds & the Logit Transformation

02

Odds Ratios & Log-Odds Ratios

03

Logistic Regression: The Model & Its Coefficients

04

The Sigmoid Function & Decision Boundaries

05

Training: MLE & Log-Loss

Evaluation & Regularization

01

Evaluating Fit: Pseudo R^2 , Deviance & P-Values

02

Saturated Models, Deviance Statistics & Residuals

03

Classification Evaluation: ROC, AUC & Confusion Matrix

04

Regularization: Ridge, Lasso & Elastic Net

05

Early Stopping & Practical Implementation

Odds, Log-Odds & the Logit Transformation

The language of chance that makes logistic regression possible

The Intuition

Most people understand **probability**: the chance of something happening out of everything that could happen. But statisticians prefer **Odds**. If your team wins 1 game and loses 4, the probability of winning is $1/5 = 0.2$, but the odds are 1 to 4 (or 0.25).

Odds range from 0 to infinity but are **asymmetric**. The logarithm fixes this — taking $\log(\text{odds})$ stretches the scale so that 'for' and 'against' are mirror images around zero. An odds ratio of 6 becomes $\log(6) = +1.79$, while $1/6$ becomes -1.79 . Beautiful symmetry.

 **Analogy:** Think of odds like a see-saw. Probability is the height of one side. Odds is the ratio of the heavy side to the light side. Log-odds is the tilt angle from centre — equal distances left or right from horizontal.

The Mathematics

Probability

$$P(\text{event}) = \text{successes} / \text{total outcomes}$$

Odds

$$\text{Odds} = p / (1 - p)$$

Log-Odds (Logit)

$$\text{logit}(p) = \log(p / (1 - p))$$

 When $p = 0.5$, $\log\text{-odds} = 0$ (the neutral point). As $p \rightarrow 1$, $\log\text{-odds} \rightarrow +\infty$. As $p \rightarrow 0$, $\log\text{-odds} \rightarrow -\infty$. This infinite range on both sides is exactly what we need for a linear model.

 **Common Mistake:** If a horse wins 3 out of 10 races, probability = 0.3 but odds = 3:7 (0.43). These are NOT the same number.



Probability

Ranges 0 to 1

Odds

Ranges 0 to ∞

Log-Odds

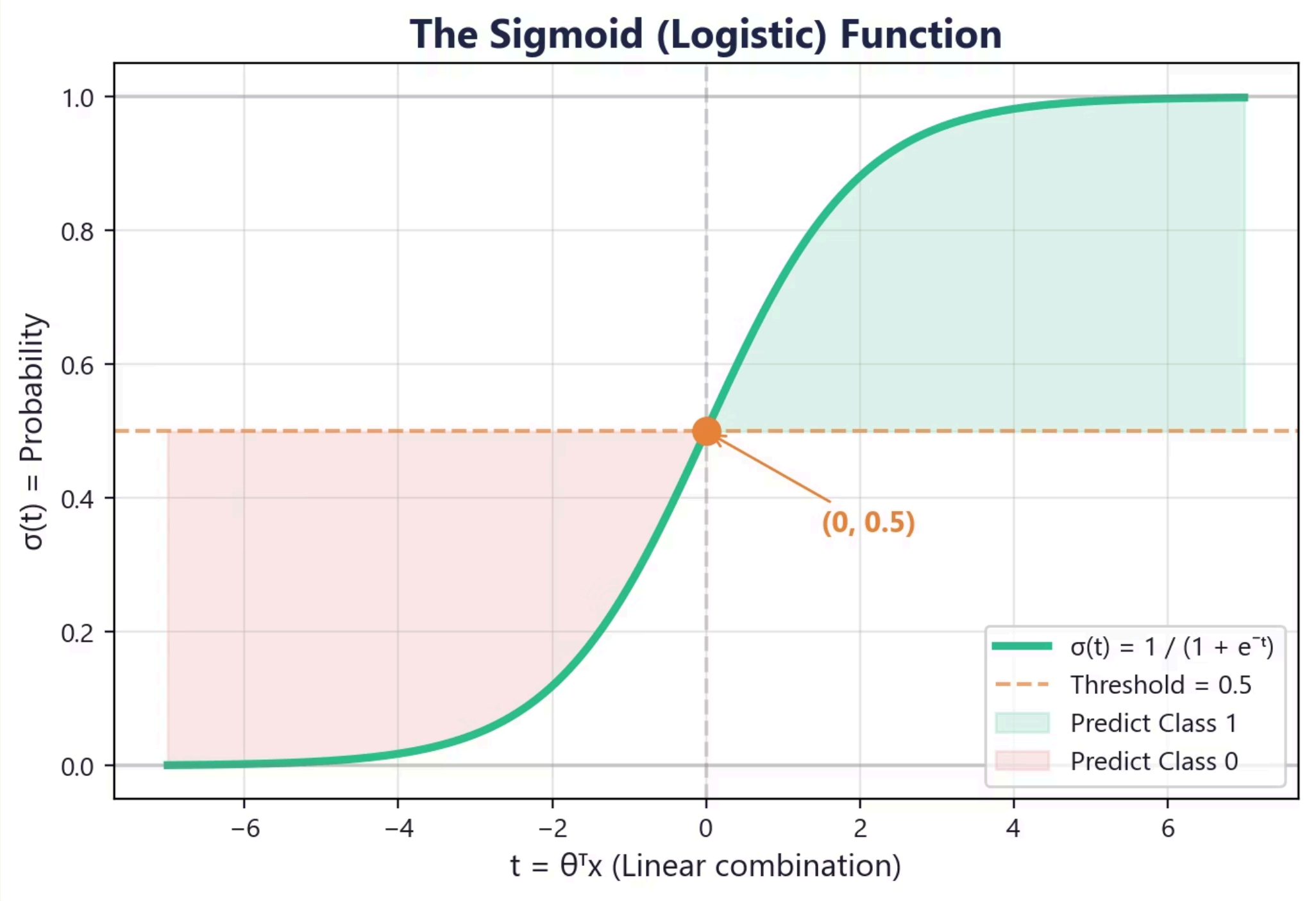
Ranges $-\infty$ to $+\infty$, symmetric around 0

Logit(0.5)

= 0, the decision point

Odds Ratios, Logistic Regression & the Sigmoid

From comparing groups to building the S-shaped model at the heart of classification.



Odds Ratios — Comparing Two Groups

An **odds ratio** compares the odds of an outcome between two groups. OR = 1 means no difference. OR = 6.88 means the odds are nearly 7× higher in one group.

Odds Ratio	Log-Odds Ratio
$OR = \frac{\text{Odds}(\text{Group 1})}{\text{Odds}(\text{Group 2})}$	$\log(OR) = \log(\text{Odds1}) - \log(\text{Odds2})$

Worked Example: 140 people with a mutated gene: 23 had cancer (odds = 23/117 = 0.196). 216 without mutation: 6 had cancer (odds = 6/210 = 0.028). OR = 0.196/0.028 = **6.88**. Log-OR = log(6.88) = 1.93.

- ✔

The logistic regression coefficient for a categorical variable IS the log-odds ratio. Exponentiate it ($e^{1.93} = 6.88$) to get the odds ratio.

The Logistic Regression Model

Logistic regression is a **Generalized Linear Model (GLM)** using the logit as its link function. It predicts $P(\text{class} = 1)$ using a sigmoid applied to a linear combination of features.

Log-Odds Form
$\log(p/(1-p)) = b_0 + b_1 \cdot x_1 + \dots + b_n \cdot x_n$

Sigmoid Function
$\sigma(t) = 1 / (1 + e^{-(t)})$

Probability from Features
$P(y=1 \mid x) = 1 / (1 + e^{-(b_0 + b_1 \cdot x)})$

⚠

Common Mistake: A slope of 1.825 does NOT mean probability increases by 1.825 per unit. It means LOG-ODDS increase by 1.825. The effect on probability depends on where you are on the S-curve.

$\sigma(0) = 0.5$
The midpoint — default decision threshold

$e^{(\text{coefficient})}$
Converts log-odds to odds ratio

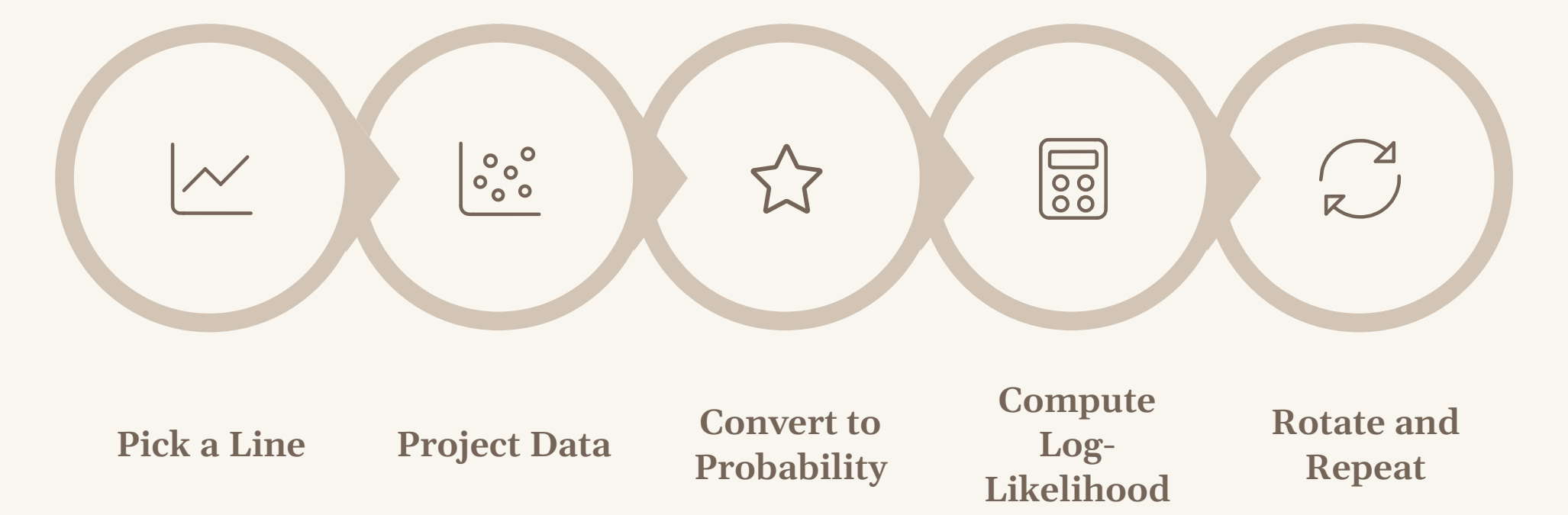
Linear Boundaries
Logistic regression always produces linear decision boundaries

C in Scikit-Learn
Inverse of regularization strength — higher C = less regularization

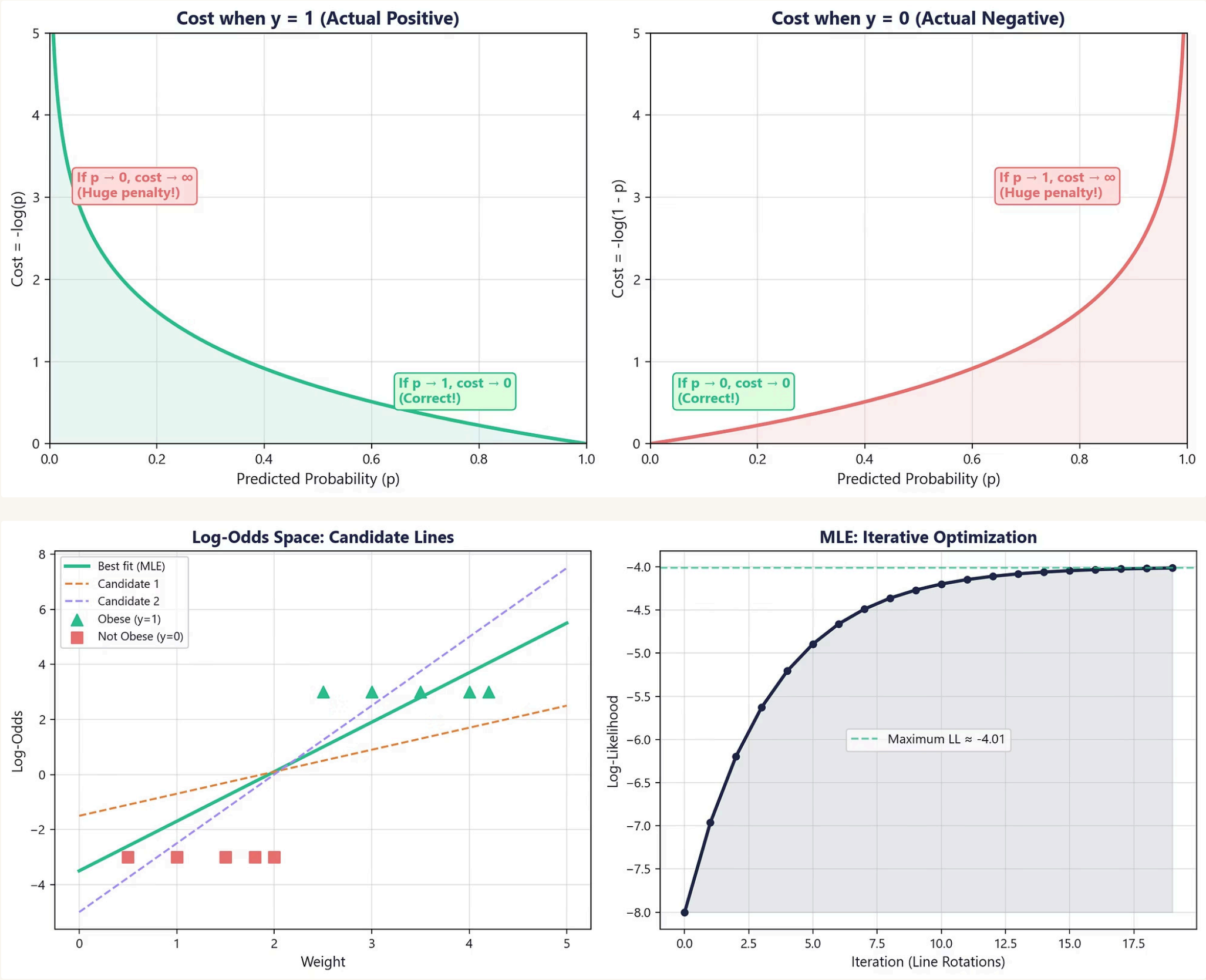
Training: Maximum Likelihood Estimation & Log-Loss

How the model learns — and why we cannot use least squares

❌ **Why Not Least Squares?** When we transform binary outcomes (0s and 1s) to log-odds space, 0 becomes $-\infty$ and 1 becomes $+\infty$, making residuals infinitely large. Least squares simply does not work for logistic regression.



MLE finds the S-curve that makes the observed data most probable. For each data point, if it is class 1, the model wants a high predicted probability; if class 0, a low probability. The algorithm maximises the product of all these 'correct' probabilities.



The Mathematics

Likelihood (one observation)

$$L = p^y \cdot (1-p)^{(1-y)}$$

Log-Likelihood

$$LL = \sum [y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

Log-Loss (Cross-Entropy)

$$J(\theta) = -(1/m) \cdot \sum [y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

Gradient

$$\partial J / \partial \theta_j = (1/m) \cdot \sum [(\sigma(\theta^T x) - y) \cdot x_j]$$

Key Insights

⚡ **Key Insight:** Maximising log-likelihood and minimising log-loss (cross-entropy) are mathematically identical. StatQuest presents MLE; Geron presents log-loss. Both arrive at the same optimal parameters.

🔍 **Analogy:** MLE is like a detective asking: 'Given the evidence, which suspect's story makes the evidence most likely?' You score each version of events and pick the highest-scoring story.

The log-loss cost function is **convex**, meaning gradient descent is guaranteed to find the global minimum. There is no closed-form solution — iterative optimisation (batch, stochastic, or mini-batch gradient descent) is required.

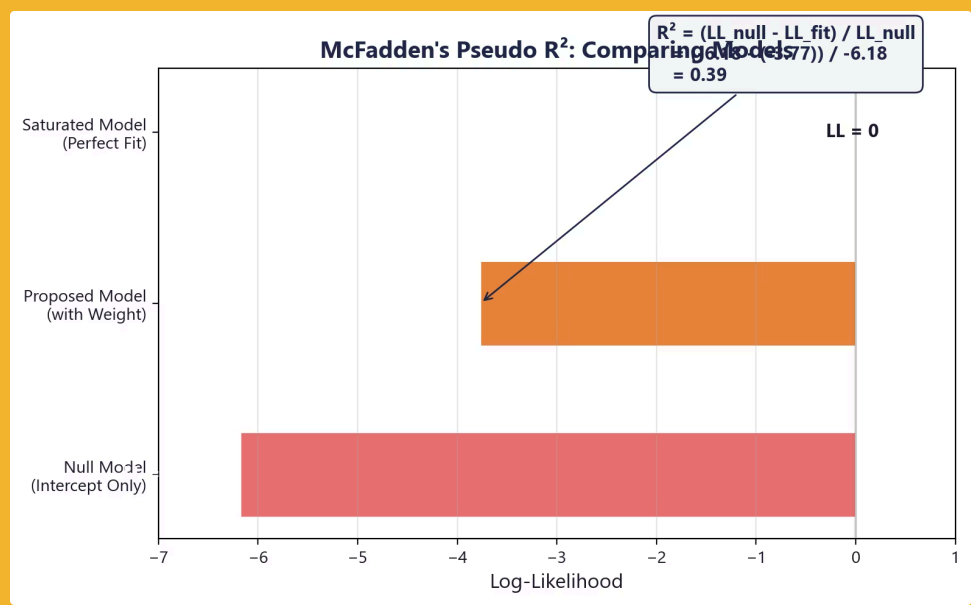
Made with GAMMA

Evaluating Fit & Deviance Statistics

Is the model actually useful? Measuring goodness of fit through pseudo R^2 , deviance, and the hierarchy of model benchmarks.

McFadden's Pseudo R^2

Traditional R^2 relies on residuals — which we do not have in logistic regression. McFadden's Pseudo R^2 compares the log-likelihood of our fitted model to a baseline null model.



Formula

$$R^2 = \frac{LL_{null} - LL_{fit}}{LL_{null}}$$

Chi-Squared Test

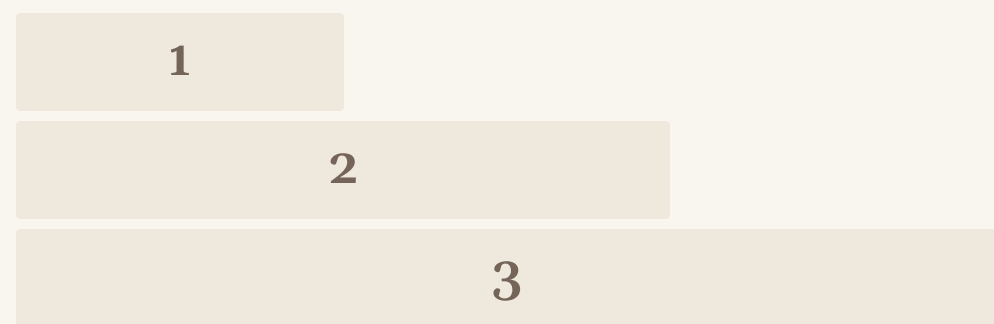
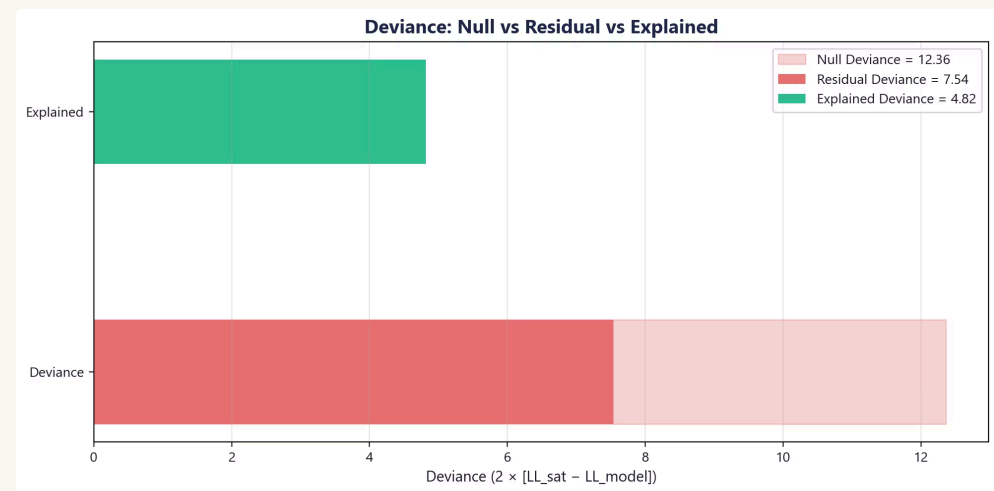
$$\chi^2 = 2 \cdot (LL_{fit} - LL_{null})$$

Worked Example: $LL(\text{fit}) = -3.77$, $LL(\text{null}) = -6.18$. $R^2 = \frac{-6.18 - (-3.77)}{-6.18} = 0.39$. The model explains 39% of the deviance. $\chi^2 = 4.82$, $p = 0.03$ — statistically significant.

⚠ Values above 0.2 are considered 'good' in logistic regression. Do NOT expect values of 0.8+ as in linear regression.

The Deviance Hierarchy

Three benchmark models make log-likelihood interpretable:



- 1 Null Model**
Intercept-only. Predicts same probability for every observation. Worst reasonable model.
- 2 Proposed Model**
Our fitted model with predictors. Sits between null and saturated.
- 3 Saturated Model**
One parameter per data point. Perfect fit. $LL = 0$ for logistic regression.

Residual Deviance

$D_{res} = -2 \cdot LL(\text{proposed})$

Null Deviance

$D_{null} = -2 \cdot LL(\text{null})$

AIC

$AIC = \text{Residual Deviance} + 2k$

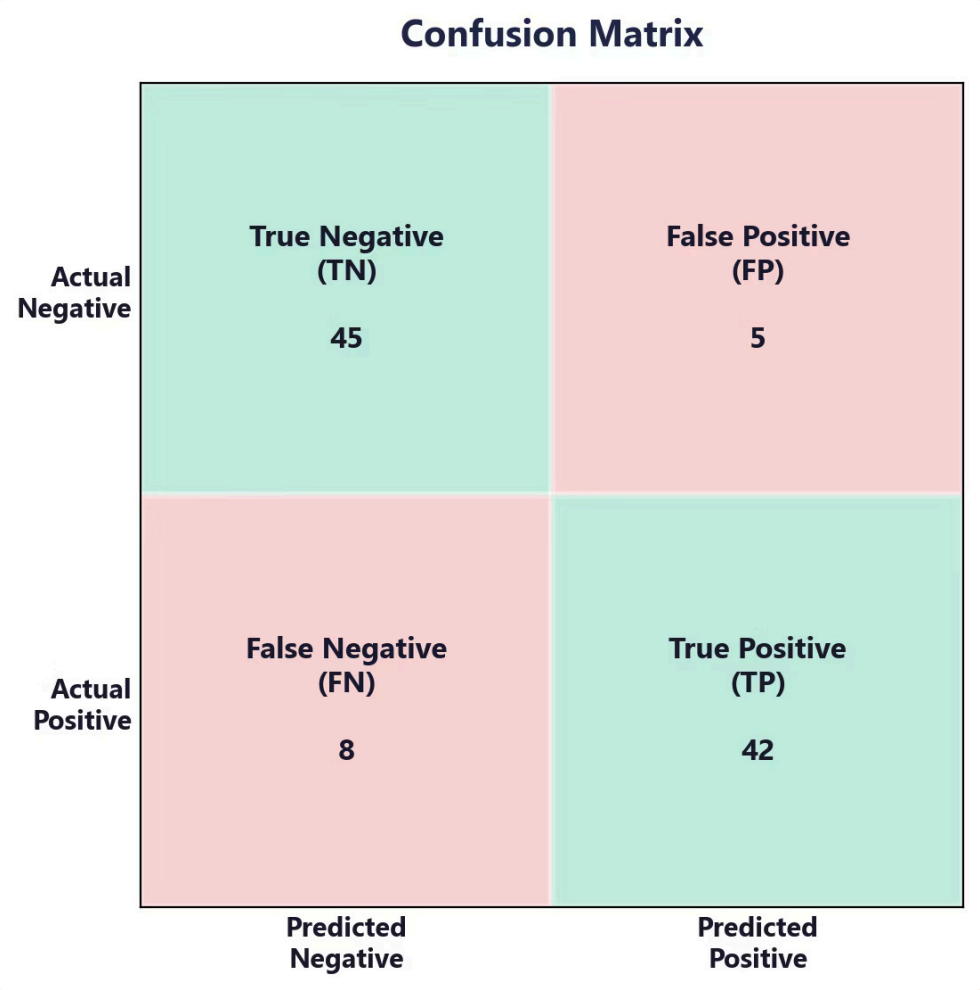
i Deviance residuals zoom in on individual observations. Points with $|d| > 2$ are potential outliers or mislabeled data.

Classification Evaluation: ROC, AUC & Confusion Matrix

Measuring how well the model actually classifies — beyond simple accuracy

The Confusion Matrix

Every classification outcome falls into one of four categories. The diagonal (TN and TP) represents correct predictions; the off-diagonal (FP and FN) represents errors.



	Predicted Positive	Predicted Negative
Actual Positive	✔ True Positive (TP)	✘ False Negative (FN)
Actual Negative	✘ False Positive (FP)	✔ True Negative (TN)

Sensitivity (TPR / Recall)

$TPR = TP / (TP + FN)$ — Of all actual positives, what fraction did we catch?

False Positive Rate

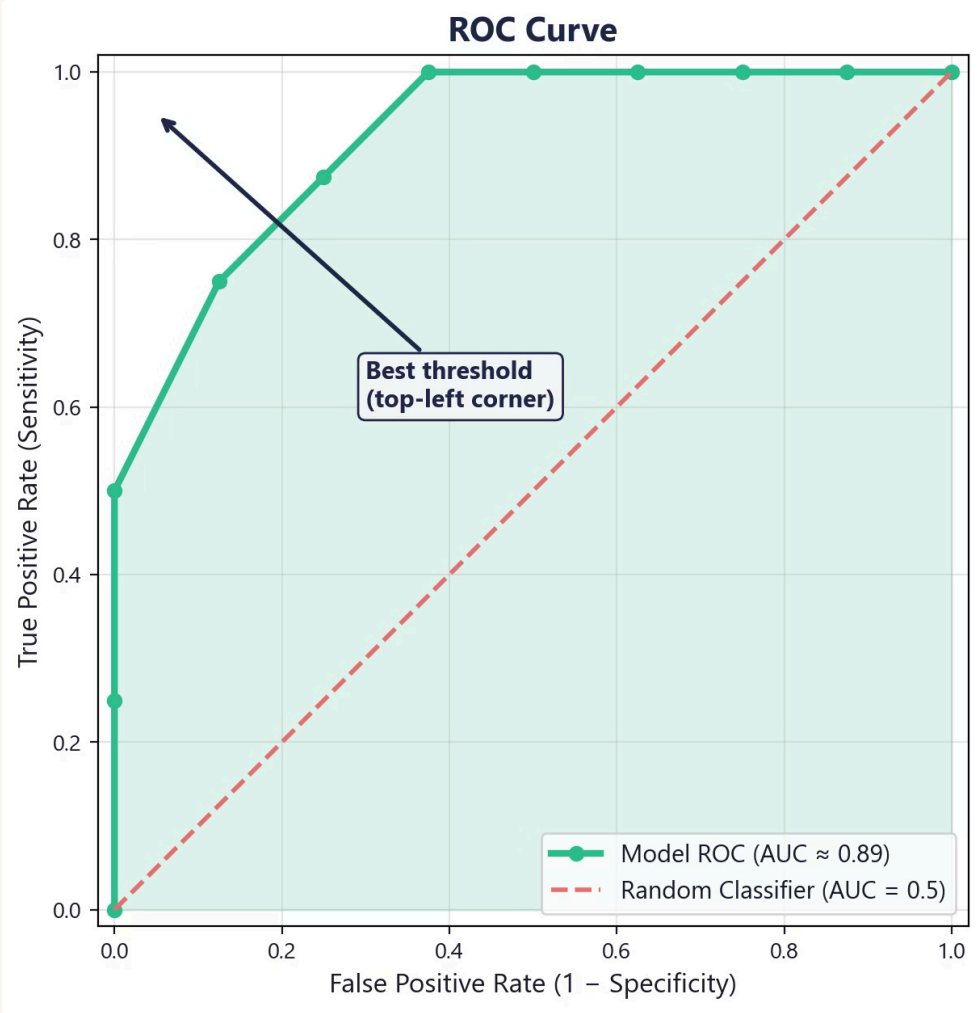
$FPR = FP / (FP + TN)$ — Of all actual negatives, what fraction did we flag?

Precision

$Precision = TP / (TP + FP)$ — Of all positive predictions, what fraction were correct?

ROC Curve & AUC

ROC curves evaluate the model at ALL possible thresholds simultaneously, showing the tradeoff between sensitivity and false positive rate. Plot FPR on x-axis, TPR on y-axis.



AUC = 1.0

Perfect classifier

AUC = 0.5

Random guessing

AUC < 0.5

Worse than random (labels flipped?)

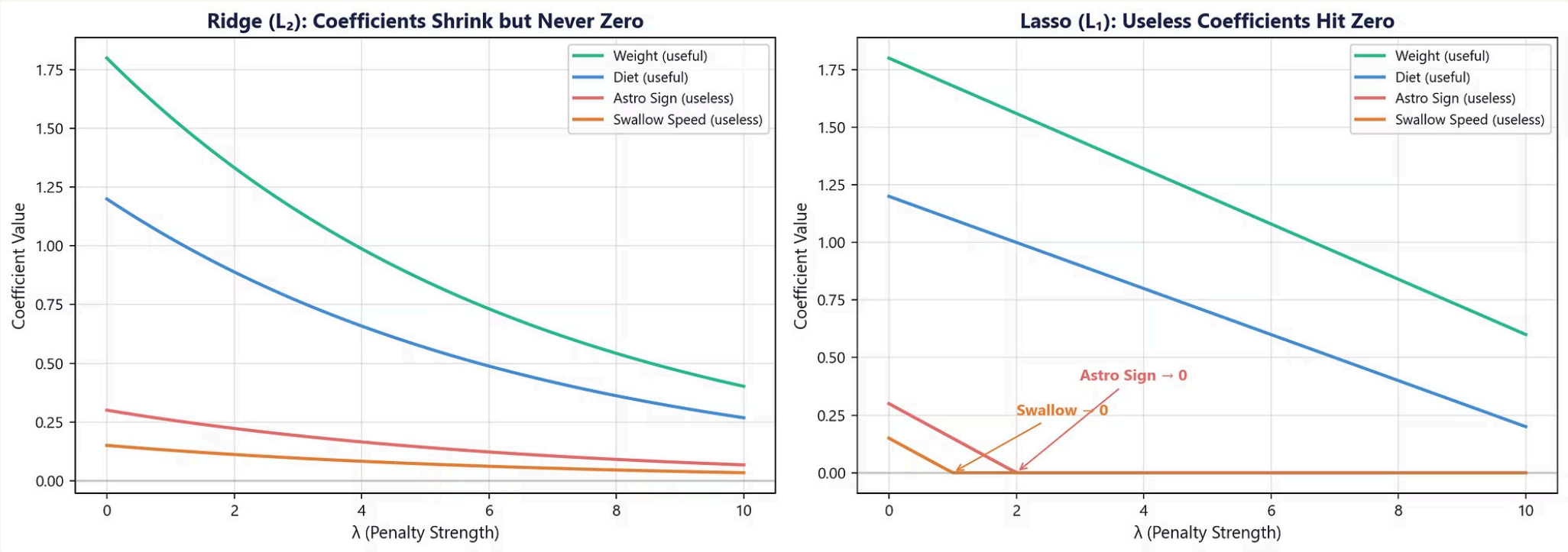
✔ ⚡ **Key Insight:** AUC = the probability that a randomly chosen positive example has a higher predicted probability than a randomly chosen negative example. AUC = 0.89 means the model correctly ranks a positive above a negative 89% of the time.

ℹ️ ✔ **Practical Tip:** Use ROC/AUC for balanced classes. For severely imbalanced datasets (e.g., 1% positive), use the Precision-Recall curve instead — high AUC can be misleading when the model achieves high specificity by predicting 'negative' most of the time.

⚠️ **Common Mistake:** Do not confuse Sensitivity (TPR) with Precision. They answer very different questions, and optimizing one often hurts the other.

Regularization: Ridge, Lasso & Elastic Net

Adding bias to reduce variance — three techniques for building models that generalize.



Ridge (L2)

Adds $\lambda \cdot \sum \theta^2$ to cost. Shrinks all coefficients toward zero but **never to exactly zero**. Best when most features are useful.

Lasso (L1)

Adds $\lambda \cdot \sum |\theta|$ to cost. Can shrink coefficients to **exactly zero** — automatic feature selection. Best when many features are useless.

Elastic Net

Combines L1 + L2: $r \cdot \alpha \cdot L1 + (1-r)/2 \cdot \alpha \cdot L2$. Groups correlated features. Best when features may be correlated or uncertain sparsity.

Why Regularize?

With only two data points, a line passes through both perfectly — but it swings wildly with new data. This is **overfitting**: perfect training fit, terrible generalization.

Ridge regression is like putting training wheels on a bicycle. The bike can still move in the right direction, but it cannot tip over at extreme angles. You sacrifice a tiny bit of agility for a huge gain in stability.

Ridge Key Insight: Ridge can solve 'unsolvable' problems. When you have more parameters than data points ($p \gg n$), ordinary least squares has infinitely many solutions. Ridge forces a unique solution by making the cost function strictly convex.

Lasso's Geometric Secret

Why does Lasso hit exactly zero while Ridge does not?

Ridge (L2) Geometry

Smooth parabola / circular constraint region. The minimum slides toward zero but never reaches it — no sharp corners.

Lasso (L1) Geometry

V-shape with a sharp kink at zero / diamond constraint region. Loss contours hit the diamond at a corner where coordinates are exactly zero.

Lasso Weakness: Struggles with correlated features — tends to arbitrarily pick one and zero out the other. Use Elastic Net instead when features are correlated.

Elastic Net Grouping Effect: Correlated features are kept or removed together, not arbitrarily split. Critical in genomics where many genes are co-regulated.

Method	Penalty	Feature Selection?	Handles Correlation?	Best Use Case
Ridge (L2)	$\lambda \cdot \sum \theta^2$	No (shrinks only)	Yes	Most features are useful
Lasso (L1)	$\lambda \cdot \sum \theta $	Yes (zeros out)	No (picks one)	Many useless features
Elastic Net	L1 + L2 mix	Yes	Yes (groups them)	Correlated features, uncertain sparsity

Early Stopping

"A beautiful free lunch" — regularization through training duration, not cost modification

The Intuition

As you train a model with gradient descent, both training error and validation error decrease at first. But after some point, training error keeps decreasing while **validation error starts to increase**. This is the moment overfitting begins.

Early stopping says: **stop training at that exact point**. Geoffrey Hinton famously called this a **"a beautiful free lunch"** because it gives you regularization without any additional computation or hyperparameters to tune.

Early stopping limits model complexity by limiting how far the optimiser can move from the initial parameter values. Since parameters start near zero, stopping early keeps them 'close to zero' — similar in effect to explicit L2 regularization.

Implementation

Monitor Validation Error

Track validation loss at each epoch during training.

Use Patience

With noisy SGD/mini-batch curves, stop only after validation error has been above the minimum for N consecutive epochs.

Roll Back

Restore the model to the best checkpoint after stopping.

i In Scikit-Learn, set `warm_start=True` in `SGDRegressor` to continue training from where you left off each `fit()` call.

No Extra Cost

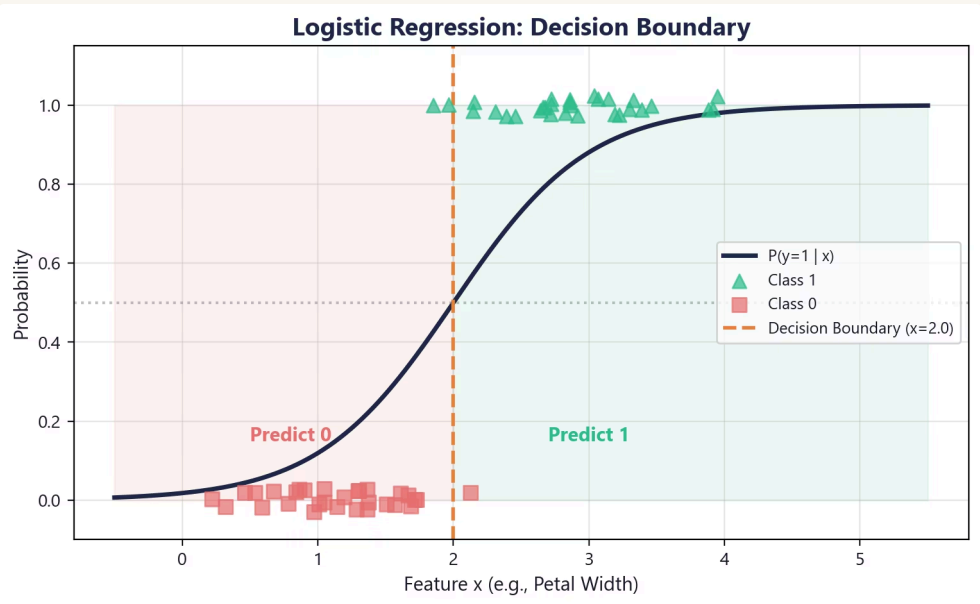
Free regularization — no penalty terms added

Implicit L2

Equivalent effect to L2 regularization

Universal

Used in virtually every neural network pipeline



Practical Implementation & Master Glossary

From theory to code — and a complete reference of all key terms, definitions, and formulas.

Python (Scikit-Learn)

```
from sklearn.linear_model import (
    LogisticRegression,
    Ridge, Lasso, ElasticNet
)

# Logistic Regression (L2 by default)
log_reg = LogisticRegression(
    C=1.0, penalty='l2'
)
log_reg.fit(X_train, y_train)
y_pred = log_reg.predict(X_test)
y_proba = log_reg.predict_proba(X_test)

# Regularized regression models
ridge = Ridge(alpha=1.0).fit(X, y)
lasso = Lasso(alpha=0.1).fit(X, y)
enet = ElasticNet(
    alpha=0.1, l1_ratio=0.5
).fit(X, y)
```

⚠ C is the INVERSE of regularization strength. Higher C = less regularization. This is the opposite of alpha used in Ridge/Lasso.

R (glm)

```
model <- glm(
  Outcome ~ Predictor,
  data = df,
  family = 'binomial'
)
summary(model)

# McFadden's Pseudo R-squared
1 - (model$deviance /
      model$null.deviance)

# Compare models by AIC
AIC(model1, model2)
```

📄 The 'binomial' family triggers the logit link function and MLE for binary data. Fisher Scoring Iterations > 25 may indicate convergence issues.

Data Preparation Checklist

01

Clean Data

Convert categories to factors; handle missing values (NA / imputation)

02

Scale Features

Use StandardScaler before any regularized model — penalties are sensitive to feature scale

03

Verify Balance

Use xtabs() in R to confirm both outcomes exist across all categories

04

Evaluate & Validate

Check deviance, AIC, Pseudo R², p-values, and plot predicted probabilities vs actuals

Master Glossary

Term	Definition	Formula / Key Expression
Probability (p)	Chance of event out of all outcomes	p = successes / total
Odds	Ratio of success to failure	p / (1 – p)
Log-Odds (Logit)	Symmetric transformation of odds	log(p / (1–p))
Odds Ratio (OR)	Comparison of two groups' odds	Odds ₁ / Odds ₂
Sigmoid Function	S-shaped curve mapping ℝ → (0,1)	1 / (1 + e ^{^(–t)})
Log-Loss (Cross-Entropy)	Negative average log-likelihood	–(1/m)·Σ[y·log(p)+(1–y)·log(1–p)]
McFadden’s Pseudo R ²	Improvement over null model	(LL _{null} – LL _{fit}) / LL _{null}
Residual Deviance	Model vs saturated gap	–2 · LL(proposed)
AIC	Deviance + complexity penalty	Residual Deviance + 2k
Sensitivity (TPR)	Fraction of positives correctly caught	TP / (TP + FN)
Precision	Fraction of positive predictions correct	TP / (TP + FP)
AUC	Area Under ROC Curve	0.0 (worst) to 1.0 (perfect)
Ridge Penalty (L2)	Sum of squared coefficients	λ · Σθ ²
Lasso Penalty (L1)	Sum of absolute coefficients	λ · Σ θ
Elastic Net	L1 + L2 hybrid	r·α·L1 + (1–r)/2·α·L2
C (Scikit-Learn)	Inverse regularization strength	C = 1/alpha
Early Stopping	Stop training at validation minimum	Monitor val_error each epoch